

# Interfaces in



# C#.NET

Part-2

Mr. Bangar Raju

 **NARESH**  
technologies

## Understanding Multiple Inheritance in C# Interfaces

By Albert Harmon

Published August 4, 2017

🕒 3 min read

Education



**TLDR** This blog post explores the concept of interfaces in C#, focusing on how multiple inheritance is supported through interfaces, the ambiguity issues with class inheritance, and practical implementation examples.

In the previous video on interfaces, we discussed the definition of an interface, how to define it, and how to implement its abstract methods in child classes. This post will delve deeper into the concept of multiple inheritance in C# and how it is uniquely supported through interfaces.

## What is an Interface?

An interface in C# is a user-defined type similar to a class but can only contain abstract method declarations. It serves as a contract that classes can implement, ensuring that they provide specific functionality.

## Defining an Interface

To define an interface, you use the following syntax:

```
interface InterfaceName
{
    // Abstract method declarations
}
```

## Types of Inheritance in Object-Oriented Programming

In object-oriented programming, there are five types of inheritance:

1. Single Inheritance
2. Multilevel Inheritance
3. Hierarchical Inheritance
4. Multiple Inheritance
5. Hybrid Inheritance

In C#, only single, multilevel, and hierarchical inheritance are supported through classes. However, multiple and hybrid inheritance are not supported due to potential ambiguity issues.

## **Multiple Inheritance and Ambiguity Issues**

### **Why is Multiple Inheritance Not Supported in Classes?**

Multiple inheritance is not supported in C# classes because it can lead to ambiguity problems. For instance, if a class inherits from two parent classes that have methods with the same name and signature, it creates confusion about which method to invoke.

### **How Interfaces Solve This Problem**

Unlike classes, interfaces do not cause ambiguity because they do not provide method implementations. Instead, they require the implementing class to define the methods. This means that if a class

implements two interfaces with the same method name, it must provide a single implementation, thus avoiding ambiguity.

## Practical Implementation of Multiple Inheritance with Interfaces

To illustrate how multiple inheritance works with interfaces, let's consider a practical example.

### Step 1: Define Interfaces

First, we define two interfaces:

```
interface InterfaceOne
{
    void Test();
}

interface InterfaceTwo
{
    void Test();
}
```

### Step 2: Implement Interfaces in a Class

Next, we create a class that implements both interfaces:

```
class MultipleInheritanceTest : InterfaceOne, InterfaceTwo
{
    public void Test()
    {
        Console.WriteLine("Interface method implemented in chi
    }
}
```

## Step 3: Calling the Method

When we create an instance of `MultipleInheritanceTest` and call the `Test` method, it will execute the implementation provided in the class without any ambiguity:

```
MultipleInheritanceTest obj = new MultipleInheritanceTest();
obj.Test(); // Output: Interface method implemented in child c
```

## Explicit Implementation

Alternatively, you can implement methods explicitly to avoid ambiguity:

```
class MultipleInheritanceTest : InterfaceOne, InterfaceTwo
{
```

```
void InterfaceOne.Test()
{
    Console.WriteLine("Test method from InterfaceOne.");
}

void InterfaceTwo.Test()
{
    Console.WriteLine("Test method from InterfaceTwo.");
}
}
```

In this case, you would need to create references to the interfaces to call their methods:

```
InterfaceOne i1 = obj;
InterfaceTwo i2 = obj;
i1.Test(); // Calls InterfaceOne's Test method

i2.Test(); // Calls InterfaceTwo's Test method
```

## Conclusion

In summary, while C# does not support multiple inheritance through classes due to ambiguity issues, it allows multiple inheritance through interfaces. This enables a class to implement multiple interfaces without

confusion, as interfaces only define method signatures without providing implementations. Understanding this concept is crucial for effective programming in C#.

For more insights and tutorials on C#, subscribe to our channel and stay updated with the latest content.